Lab 8 (for Bonus): Adversarial Machine Learning

Course: CSCI-4341-04-Fall 2025 Foundations of Intelligent Security Systems

Due date to submit the report: 12/15/2025, 11:59 PM

1. Introduction & Goal

The goal of this final lab is to explore the field of adversarial machine learning. While modern AI models can achieve superhuman performance on many tasks, they are often surprisingly fragile. This assignment will use image classification as a straightforward example to demonstrate how these models can fail. You will first train a neural network to classify handwritten digits with high accuracy. Then, you will subject this model to a series of "attacks", ranging from natural data degradation to sophisticated, purpose-built adversarial examples, and analyze the resulting impact on the model's performance.

Note on Code: All the necessary Python code for this assignment is provided to you in the accompanying Google Colab notebook. You do not need to write the neural network architecture or training loops from scratch. Your role is to execute the code, analyze the outputs, and "play" with specific parameters (instructions will be provided below) to understand their effects on the model.

2. Background: From Pixels to Predictions

How a Computer "Sees" an Image

To a computer, an image is not a picture – it is a grid of numbers. For a simple grayscale image, each number represents the brightness of a single pixel, typically ranging from 0 (black) to 255 (white). A 28x28 pixel image is therefore represented as a 28x28 matrix of these numbers.

Code Example of Raw Pixel Data (0-255 range):

```
# A small slice of a raw image matrix from the MNIST dataset
[[  0   0  18  85 155]
 [  0   0 124 253 253]
 [  0   0  18 238 253]
 [  0   0  82 253 253]
 [  0   0  22 180 253]]
```

Normalization is a critical preprocessing step where we scale these values to a consistent range, typically 0.0 to 1.0, by dividing by 255. This ensures that all input features are treated equally by the model during training, leading to faster and more stable learning.

Gaussian Noise: Simulating Real-World Imperfection

In the real world, data is rarely perfect. Cameras have sensor grain, low-light conditions introduce static, and transmission errors corrupt bits. In machine learning, we simulate this natural degradation using **Gaussian Noise** (also known as white noise).

Unlike adversarial attacks, which are calculated to trick the model, Gaussian noise is random. We add a random value to every pixel in the image. These random values are drawn from a *Gaussian*

(Normal) Distribution, which is the classic “Bell Curve”.

The formula for adding noise is:

$$x_{noisy} = \text{clip}(x_{original} + \mathcal{N}(\mu, \sigma^2), 0, 1)$$

Where:

- $x_{original}$ is the original pixel value (0.0 to 1.0).
- $\mathcal{N}$ represents the Normal Distribution.
- μ (mu) is the **Mean**, typically set to 0. This means the average change to a pixel is zero; some get brighter, some get darker.
- σ (sigma) is the **Standard Deviation**. This controls the **intensity** or “loudness” of the noise. A higher σ means the random values added are larger, making the image grainier.
- **clip(..., 0, 1)** ensures that after adding noise, the pixel value does not go below 0 (pure black) or above 1 (pure white).

Neural Network Fundamentals

A neural network is composed of layers of interconnected “neurons”. Information flows from the input layer, through one or more “hidden” layers, to the output layer.

- **Flatten Layer:** A simple reshaping layer that takes the 2D grid of pixels (e.g., 28x28) and converts it into a 1D list of numbers (e.g., a vector of 784 pixels).

  Code Example of a Flattened 1D Vector:

```
# The 28x28 grid is unrolled into a single long list of 784 numbers.
[0.0, 0.0, 0.0, ..., 0.98, 0.59, 0.0, ..., 0.0]
```

- **Dense Layer:** The core processing layer. Every neuron in a dense layer is connected to every neuron in the previous layer. Each connection has a **weight (w)** and each neuron has a **bias (b)**. A neuron calculates a weighted sum of its inputs, adds its bias, and then passes the result through a non-linear **activation function** (like ReLU) to produce its output.

The formula for the output of a single neuron is:

$$\text{output} = \text{activation}\left(\left(\sum_{i=1}^{n} w_i x_i\right) + b\right)$$

Plain Language Explanation: This formula calculates a “weighted score”. It multiplies each input value (x_i) by its importance (its weight, w_i), sums these up, adds a small offset (the bias, b), and finally passes this total score through an activation function to decide the neuron’s final output.

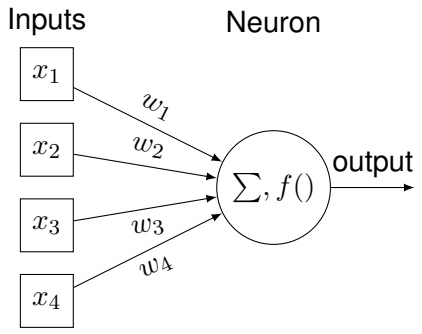


Figure 1: Flow of information into a single neuron. Symbols: x_i (input values), w_i (weights), $\sum$ (summation), $f()$ (activation function).

Model Architecture

The specific sequence of layers defines the model's architecture. For this lab, we use a simple but effective architecture:

1. **Input Layer:** A `Flatten` layer that takes the 28x28 image and converts it into a 784-node vector.
2. **Hidden Layer:** A `Dense` layer with 128 neurons and a ReLU activation function. This is where the model learns complex patterns from the pixel data.
3. **Output Layer:** A final `Dense` layer with 10 neurons, one for each digit (0-9). This layer produces the final classification scores.

The Loss Function and Training

The **loss function** measures how "wrong" the model's prediction is compared to the correct answer. The **training target** is to adjust the model's weights and biases to **minimize this loss**.

For classification, we use **Cross-Entropy Loss**. It compares the model's predicted probability distribution (generated by a `softmax` function on the output layer) to the true distribution (where the correct class is 1 and all others are 0).

The formula for Cross-Entropy Loss is:

$$\text{Loss} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

Plain Language Explanation: This formula calculates a penalty score. For the single correct class, y_i is 1. If the model's predicted probability for that class ($\hat{y}_i$) is low (close to 0), its logarithm is a large negative number, and the negative sign in front makes the overall Loss very high. The goal of training is to make $\hat{y}_i$ as close to 1 as possible for the correct class, which makes the Loss approach zero. For all incorrect classes, y_i is 0, so they do not contribute to the sum.

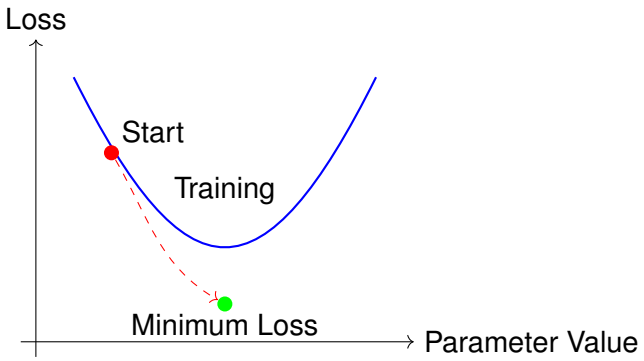
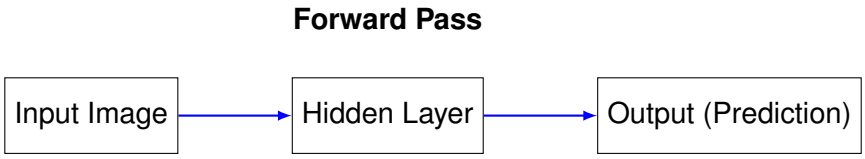


Figure 2: Training minimizes the loss function by adjusting model parameters. Symbols: C (total number of classes), y_i (true probability for class i), $\hat{y}_i$ (predicted probability for class i).

Forward and Backward Pass: How a Network Learns

Training a neural network involves a two-stage cycle that is repeated thousands of times.

1. The Forward Pass (Making a Prediction): This is the process of data flowing forward through the network. An input image is passed through the flatten layer, then the hidden layer, and finally the output layer to produce a prediction. This happens every time the model is used, both during training and testing.



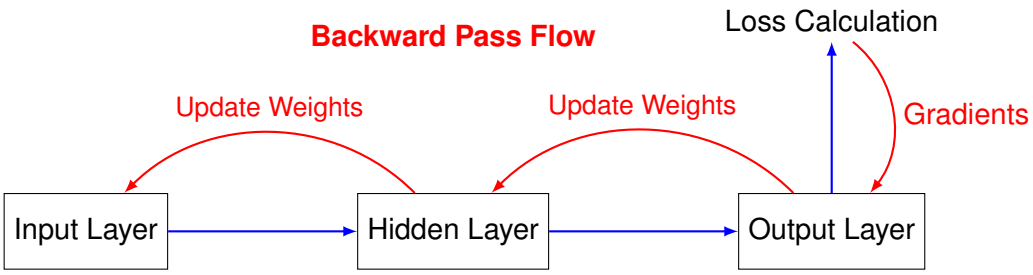
2. The Backward Pass (Learning from the Error): This process, also known as **backpropagation**, only happens during training. After the forward pass, the loss function calculates the error. The back pass then propagates this error information backward through the network, from the output layer to the input layer. As it travels, it calculates the **gradient** (the derivative of the loss with respect to each weight). This gradient tells us how much each weight contributed to the error.

The model's optimizer (e.g., 'adam') then uses this gradient to update each weight slightly in the direction that will reduce the loss.

The formula for a single weight update is:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w_{\text{old}}}$$

Plain Language Explanation: The new weight (w_{new}) is the old weight (w_{old}) minus a small step in the direction of the error. The size of the step is controlled by the learning rate (η), and the direction is determined by the gradient ($\frac{\partial L}{\partial w_{\text{old}}}$), which is the calculated contribution of that weight to the total loss (L).



Further Reading

- **TensorFlow Tutorials:** Basic image classification
- **Adversarial Attacks:** Fast Gradient Sign Method (FGSM)

6. Assignment Tasks

The code in your provided notebook is structured in a sequence of seven specific items. You are required to follow this sequence, execute the code, and provide analysis for each step in your report.

Task 1: Data Exploration

Your first task is to explore the data. Run the data loading cells in the notebook. The code will display the images in their raw numerical format (pixel values ranging from 0 to 255) and then again after normalization. In your report, visually inspect the raw pixel data and comment on how the computer interprets these digits differently than a human would.

Report Requirement: Include a screenshot or copy-paste of the raw pixel matrix output by the code. Write 1-2 sentences explaining why the values are between 0 and 255 and why normalization to 0.0-1.0 is necessary.

Task 2: Baseline Model Training

Next, you will define and train the neural network model. Execute the code block that sets up the model architecture and trains it on the original, unaffected dataset. Once the training is complete, the code will output the performance results on the test data. Record this accuracy in your report; this serves as your “Baseline Accuracy”, which you will use to compare against future experiments.

Report Requirement: State clearly the final Test Accuracy of your model on the clean data (e.g., “The baseline accuracy is 98.2%”).

Task 3: Gaussian Noise Simulation

In this step, you will simulate natural data degradation by adding Gaussian noise. Run the code to add noise to the dataset. The notebook will generate a visualization comparing the original images to the noisy images, as well as a comparison of the raw data values (original vs. noise). Analyze these differences in your report. Furthermore, you must “play” with the noise levels. Adjust the

noise factor variable in the code to different values, and observe how the raw numerical data changes.

Report Requirement: Include the visualization generated by the code that shows the "Original" digit next to the "Noisy" digit. Describe in your own words how the raw numbers in the matrix changed after adding noise.

Task 4: Evaluating Robustness to Noise

Now that you have created noisy data, you must test the resilience of your model. Take the model you trained in Task 2 (which was trained on clean data) and evaluate it on the data with Gaussian noise. You must experiment with at least three different noise levels (e.g., `noise_factor` = 0.1, 0.3, 0.5) by modifying the code variable and re-running the evaluation.

Report Requirement: Create a table with two columns: **Noise Factor** and **Model Accuracy**. Fill this table with your results from the three experiments. Discuss whether the accuracy drop was linear or if there was a specific "breaking point" where the model failed completely.

Task 5: Adversarial Attacks (FGSM)

Moving beyond random noise, you will now execute targeted adversarial attacks using the Fast Gradient Sign Method (FGSM). Run the provided code to generate these attacks. You must visualize and compare three things: the original data, the adversarial noise (perturbation) itself, and the final data affected by the attack. In your report, explain the visual difference between the random Gaussian noise from Task 3 and this targeted adversarial noise.

Report Requirement: Include the figure generated by the code showing the sequence: **Original Image** $\rightarrow$ **Perturbation (Noise)** $\rightarrow$ **Adversarial Example**.

Task 6: Evaluating Vulnerability to Attacks

With the adversarial examples generated, it is time to assess the damage. Take your original model (trained on clean data) and evaluate it on the adversarially attacked data using different values for ϵ (epsilon), which controls the attack strength.

Report Requirement: Create a table comparing **Epsilon Value** vs. **Model Accuracy**. Compare this accuracy drop to the drop you saw with Gaussian noise in Task 4. Explain why the model is more vulnerable to this targeted attack than to random noise, even if the visual distortion appears similar.

Task 7: Defense via Adversarial Training

Finally, you will attempt to defend the model. The last section of the code will train a new model that is more robust to adversarial examples. Once trained, you must test this new model on each of the three test data cohorts: the original clean data, the data with Gaussian noise, and the data with adversarial attacks.

Report Requirement: Create a final summary table comparing the **Standard Model** vs. the **Robust Model** across all three datasets (Clean, Noisy, Adversarial). Discuss if the robust model

successfully defended against the attack and if there was any trade-off (loss of accuracy) on the clean data.

7. Submission Requirements

You are required to submit two separate items:

A **PDF report** that describes all the steps of your analysis in a clear, written form. This report must include all your data visualizations, tables, and your written interpretations/conclusions for each task as specified in the “Report Requirement” sections above.